Lab 02: Interfacing Push Button and LED

ECE0202: Embedded Processors and Interfacing

Spring 2026

Stefan Zhang.....Student ID 2024141520349
Knight Liu .....Student ID 2024141520361

Date Submitted: April 6, 2026

1) Pre-Lab Assignment

The completed pre-lab assignment is attached in the following pages. It includes the required mask and value calculations for GPIO clock enabling, LED output configuration, and pushbutton input configuration.

Lab 2: Interfacing Push-button and LED in C

Pre-Lab Assignment

Student Name: 张鹤扬 刘德熙

TA: _____

Time & Date: 2026/04/03

Video/Textbook Reading

1. Textbook **Chapter 4.6** to review bit-wise operations and **Chapter 14 GPIO.**
2. Watch YouTube Tutorials (http://web.eece.maine.edu/~zhu/book/tutorials.php)
   - Lecture 5: Memory-mapped I/O (8 minutes)
   - Lecture 6: GPIO Output (11 minutes)
   - Lecture 7: GPIO Input (12 minutes)

Summary of Task: Section 1 and section 2.a show (1) how to calculate the mask and the value and (2) how to use them in C code. Follow these examples, find out the mask and value in the rest sections.

1. Enable the clock of GPIO Port A (for Green LED) and Port C (for User Button)

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AHB2ENR														RNGEN		AESEN			ADCEN	OTGFSEN					GPIOHEN	GPIOGEN	GPIOFEN	GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
Mask																														1		1
Value																														1		1

In the bitmask (or called **mask**) of a variable, **1** indicates that the corresponding bit in the variable is of interest. In this example, we are interested in bit 0 and bit 2, thus we have the mask as

Mask in hex = 0x00000005 Value in hex = 0x00000005

Our C code is like:

```
RCC->AHB2ENR &= ~(mask); // Clear all bits of interests

RCC->AHB2ENR |= value;   // Set bits of interests to target value
```

Note 1: Bit-wise Operations

Bit-wise operations update specific bits without affecting the other bits. For example, to set bit 2 of a 32-bit variable aWord, the following is incorrect because it resets all the other bits in this variable.

```
aWord = 4;    // Set bit 2, but also clear all the rest 31 bits
```

The correct one is to use the bit-wise OR operation:

```
aWord |= 4;   // Set bit 2 without affecting the rest
```

Note 2: **Why do we need a bitmask?**

A bitmask is often used for clearing the bits of interest. For example, the following code sets the least significant four bits of variable V to be 0b0101.

```
Mask = 0xF;  // Mask for least significant four bits

V &= ~Mask;  // Clearing bit 15 and bit 2

V |= 0x5;    // Set least significant four bits to 0x0101
```

The following code, which directly uses bitwise OR without clearing the target bits first, sometimes fails. (why?)

```
V |= 0x5;    // The result is not always correct.
```

2. Pin Initialization for Green LED (PA 5)

a. Set the mode of PA 5 to Output

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Mode Register (MODER)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
Mask																					1	1										
Value																					0	1										

GPIO Mode: Input (00), Output (01), Alternate Function (10), Analog (11, default)

Mask in hex = 0x00000C00 Value in hex = 0x00000400

Example: The following C code can correctly set the mode of PA 5 to output.

```
GPIOA->MODR &= ~0x00000C00; // Clear bit 10 and bit 11
GPIOA->MODR |=  0x00000400; // Set the mode to output
```

By following the above examples, find out the mask and value in the following sections.

b. Set the output type of PA 5 to Push-Pull

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OTYPER	Reserved																OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
Mask																											1					
Value																											0					

Push-Pull (0, reset), Open-Drain (1)

Mask in hex = 0x 00000020 Value in hex = 0x 00000000

c. Set PA 5 to No pull-up No pull-down

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		**PUPDR5[1:0]**		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
Mask																					1	1										
Value																					0	0										

No Pullup, No pulldown (00, reset), Pullup (01), Pulldown (10), Reserved (11)

Mask in hex = 0x 00000C00 Value in hex = 0x 00000000

3. Pin Initialization for the user pushbutton (PC 13)

a. Set the mode of pin PC 13 to Input

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Mode Register (MODER)	MODER15[1:0]		MODER14[1:0]		**MODER13[1:0]**		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
Mask					1	1																										
Value					0	0																										

GPIO Mode: Input (00), Output (01), Alternate Function (10), Analog (11, default)

Mask in hex = 0x 0C000000 Value in hex = 0x 00000000

b. Set PC 13 to no pullup and no pulldown

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		**PUPDR13[1:0]**		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		**PUPDR5[1:0]**		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
Mask					1	1																										
Value					0	0																										

No Pullup, No pulldown (00, reset), Pullup (01), Pulldown (10), Reserved (11)

Mask in hex = 0x 0C000000 Value in hex = 0x 000000

2) Experimental Results

2.1) Pushbutton and LED Functionality

In this experiment, the onboard green LED connected to pin PA5 was controlled by the blue user pushbutton connected to pin PC13.

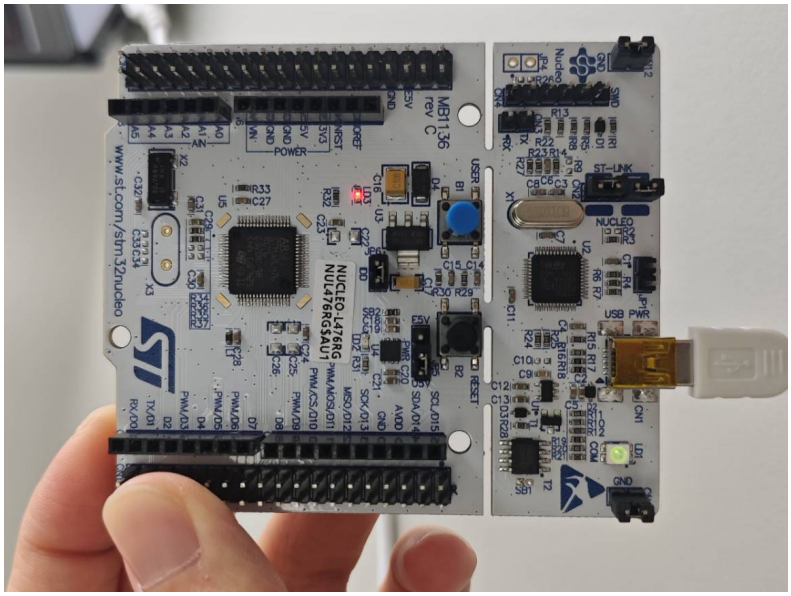

Figure 1: The green LED is turned on during the experiment.

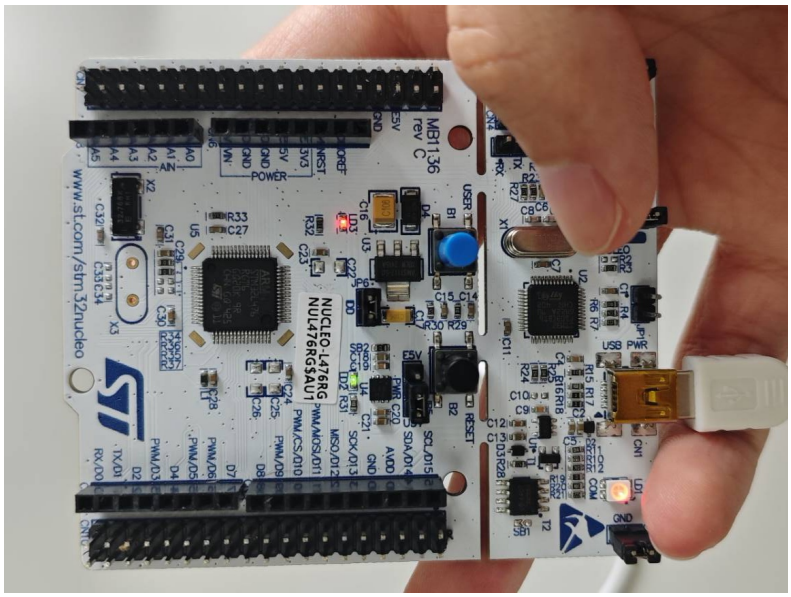

Figure 2: The green LED is turned off during the experiment.

The experimental results show that the system worked as expected. A short press changed the LED state once, while a long press caused the LED to flash continuously. This behavior is consistent with the polling-based control logic implemented in the program.

2.2) Verification of GPIO Input Register

The debug environment in Keil was used to observe the value of the GPIO input register for port C. Since the user button is connected to PC13 and is externally pulled up, the input value should be logic high when the button is not pressed and logic low when it is pressed.

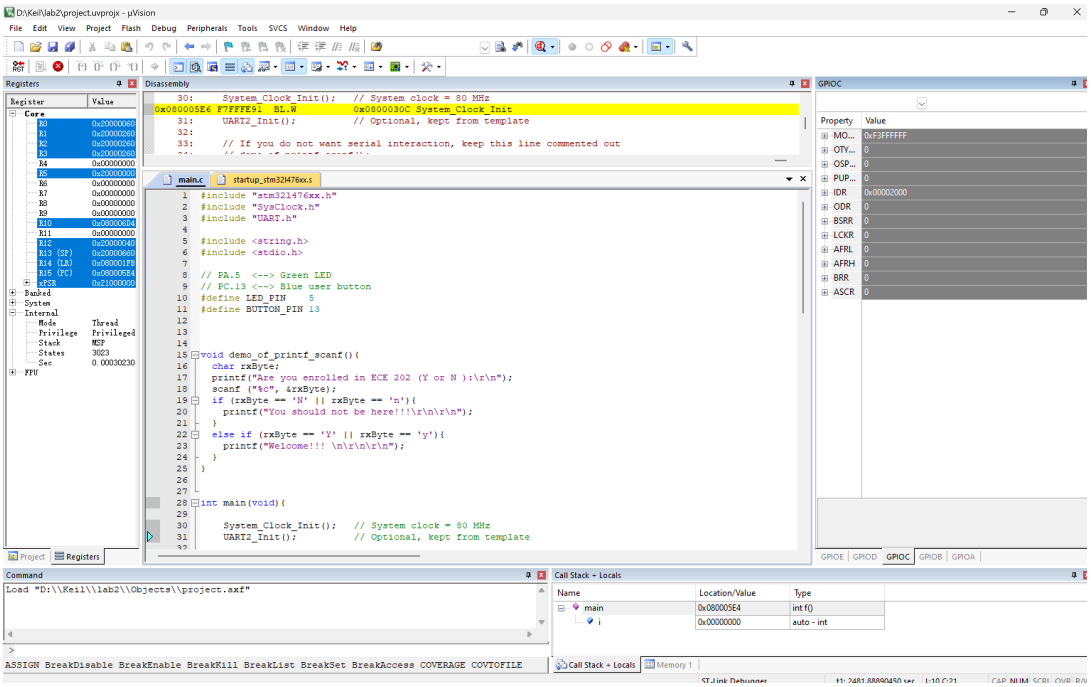

Figure 3: GPIOC input data register when the pushbutton is not pressed.

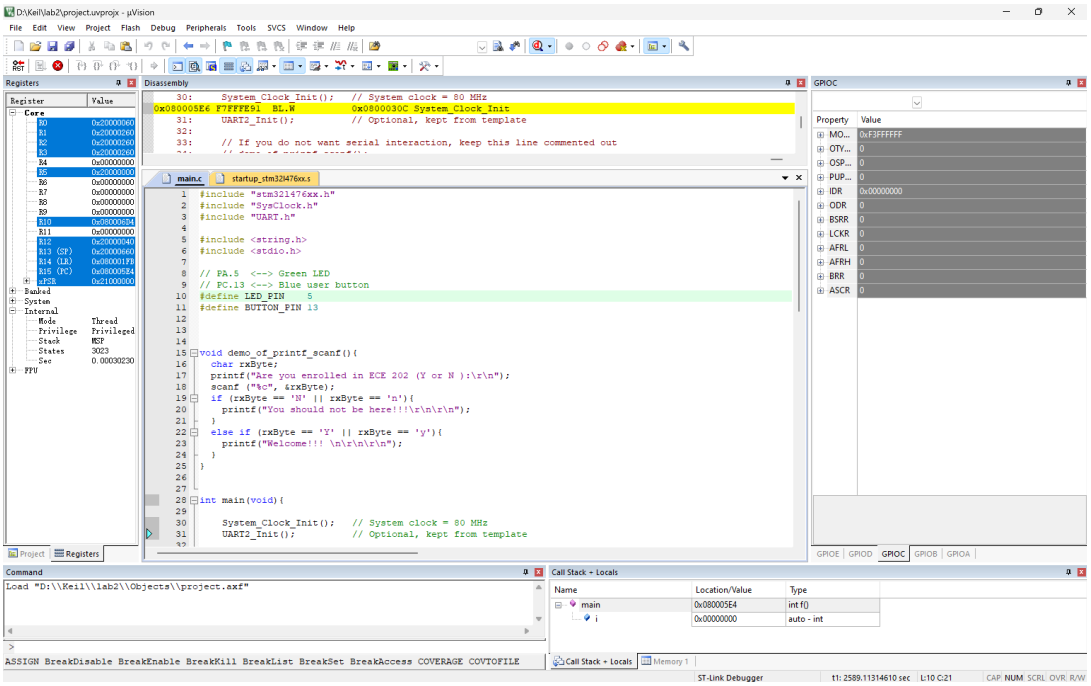

Figure 4: GPIOC input data register when the pushbutton is pressed.

The observed register values matched the expectation. When the button was not pressed, bit 13 in the input data register remained high. When the button was pressed, the value changed to low. This confirms that the button input was correctly detected by the microcontroller. These results verify that PC13 operates as an active-low input pin in this experiment.

2.3) Verification of GPIO Output Register

The output data register of GPIOA was also observed in debug mode. Because the green LED is connected to PA5, the LED state is determined by bit 5 of the GPIOA output data register.

When the LED was off, the corresponding output bit was cleared. When the LED was on, the output bit was set. Manual modification of the ODR value in the debugger also directly changed the LED state, which further confirms that PA5 was correctly configured as a digital output pin.

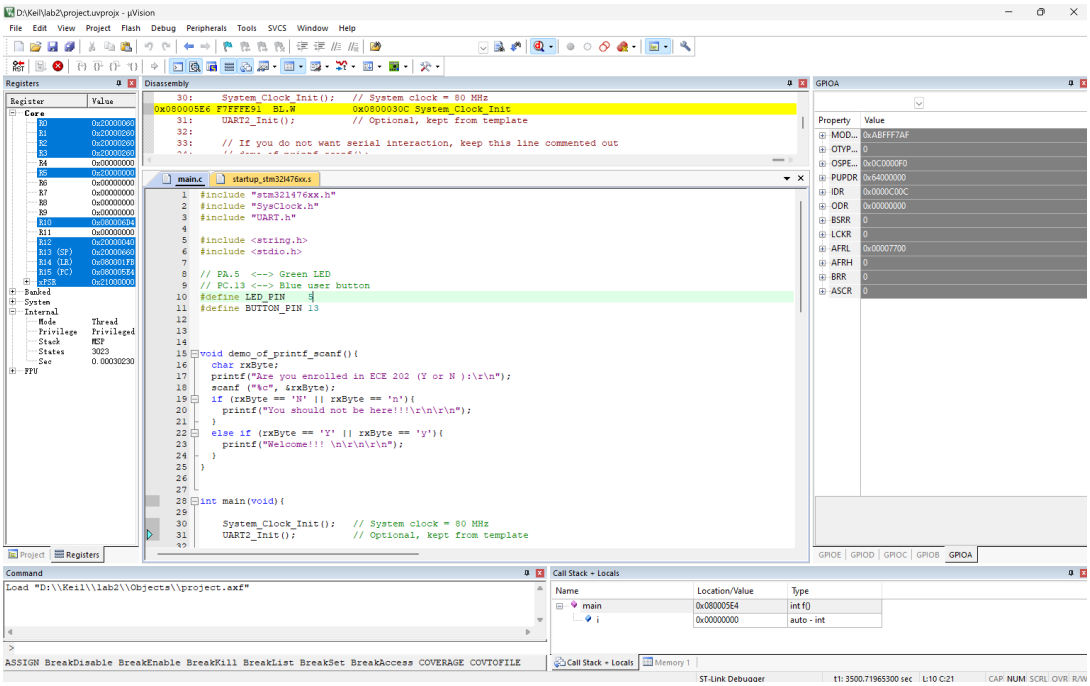

Figure 5: GPIOA output data register when the LED is off.

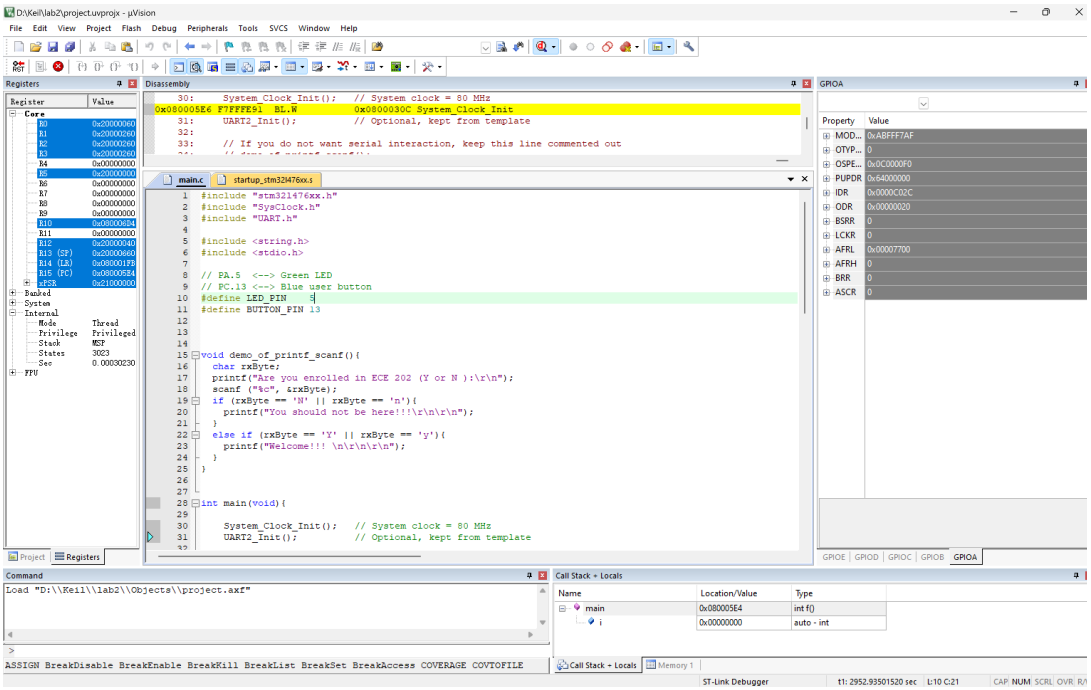

Figure 6: GPIOA output data register when the LED is on.

2.4) Discussion

The experimental behavior agrees with the program design. In the code, GPIOA and GPIOC clocks were enabled first. Then PA5 was configured as a push-pull output with no pull-up and no pull-down, while PC13 was configured as an input pin with no pull-up and no pull-down. The program continuously checked the button state through polling.

When the pushbutton was pressed, the program detected that bit 13 of `GPIOC->IDR` became 0, and then toggled bit 5 of `GPIOA->ODR`. A short delay was inserted after each toggle to make the LED flashing visible and to prevent the LED from changing too quickly.

This experiment shows the basic method of GPIO interfacing in C and also illustrates the limitation of polling I/O, since the processor repeatedly checks the button state in the infinite loop.

3) Code

The main program used in this experiment is shown below.

```
#include "stm32l476xx.h"
#include "SysClock.h"
#include "UART.h"
#include <string.h>
#include <stdio.h>
#define LED_PIN    5
#define BUTTON_PIN 13

void demo_of_printf_scanf(){
```

```
        char rxByte;
        printf("Are you enrolled in ECE 202 (Y or N ):\r\n");
        scanf ("%c", &rxByte);
        if (rxByte == 'N' || rxByte == 'n'){
                printf("You should not be here!!!\r\n\r\n");
        }
        else if (rxByte == 'Y' || rxByte == 'y'){
                printf("Welcome!!! \n\r\n\r\n");
        }
}

int main(void){
    System_Clock_Init();
    UART2_Init();

    RCC->AHB2ENR &= ~(0x00000005);
    RCC->AHB2ENR |=  (0x00000005);

    GPIOA->MODER &= ~(0x00000C00);
    GPIOA->MODER |=  (0x00000400);

    GPIOA->OTYPER &= ~(0x00000020);
    GPIOA->OTYPER |=  (0x00000000);

    GPIOA->PUPDR &= ~(0x00000C00);
    GPIOA->PUPDR |=  (0x00000000);

    GPIOC->MODER &= ~(0x0C000000);
    GPIOC->MODER |=  (0x00000000);

    GPIOC->PUPDR &= ~(0x0C000000);
    GPIOC->PUPDR |=  (0x00000000);

    GPIOA->ODR &= ~(1 << LED_PIN);

    while(1){
        if ((GPIOC->IDR & (1 << BUTTON_PIN)) == 0) {
            GPIOA->ODR ^= (1 << LED_PIN);
            toggling
            for (volatile int i = 0; i < 200000; i++);
        }
    }
}
```

4) Conclusion

In this lab, a C program was successfully developed to interface the onboard pushbutton and green LED on the STM32 NUCLEO-L476RG board. The button input on PC13 was correctly detected, and the LED output on PA5 responded as intended. The debug results also confirmed the expected changes in the GPIO input and output registers.

Overall, the experiment achieved its goal of demonstrating basic GPIO input/output configuration and simple polling-based digital control in an embedded C program.